
Assignment 5

Q1 Distribution Analysis, Skewness & Normality Testing

Task Objective

Assess the shape, spread, and normality of numerical features by combining descriptive statistics, distribution-shape metrics, formal hypothesis testing, and visual diagnostics.

Tasks

- 1 Read `customer_data.csv` into a Pandas DataFrame and display structural meta-information using `.info()` and summary statistics using `.describe()`.
- 2 Compute numerical skewness and kurtosis metrics specifically for Age and Annual_Income.
- 3 Conduct a formal Shapiro-Wilk test for normality on Age and Annual_Income using `scipy.stats.shapiro()`. Output test statistics and corresponding p-values.
- 4 Construct a 1×2 grid figure containing: Left Plot – Histogram with a KDE overlay for Age using `sns.histplot()`; Right Plot – KDE density estimation for Annual_Income using `sns.kdeplot()`.

Q2 Missing Value Detection & Imputation Strategies

Task Objective

Diagnose non-random patterns of missingness across DataFrame features and execute comparative numerical imputations.

Tasks

- 1 Tabulate total missing counts and percentage ratios across all columns in customer_data.csv.
- 2 Generate a binary missingness heatmap (`sns.heatmap(df.isnull(), cbar=False)`) to visually evaluate whether missing entries are correlated or clustered across rows.
- 3 Perform Mean Imputation on Age and Median Imputation on Annual_Income. Store these in new columns: Age_Mean_Imp and Income_Median_Imp.
- 4 Apply distance-based KNNImputer (`k = 5`) to handle missing values in Age and Annual_Income simultaneously using neighbor similarity scores.
- 5 Print comparative mean, median, and standard deviation values across all feature variants (Original, Mean/Median Imputed, KNN Imputed).

Q3 Outlier Detection, Quantification & Capping

Task Objective

Identify extreme statistical anomalies using non-parametric (IQR) and parametric (Z-Score) boundary methods, then treat them via Winsorization (capping).

Tasks

- 1 Determine the exact Interquartile Range (IQR) for Annual_Income. Extract and print all records where values fall below $Q1 - 1.5 \times IQR$ or exceed $Q3 + 1.5 \times IQR$.
- 2 Calculate Z-scores for Spending_Score using `scipy.stats.zscore()` and filter out all records where $|Z| > 3.0$.
- 3 Create a two-panel side-by-side box plot figure comparing raw distributions of Annual_Income and Spending_Score against their outlier-cleaned representations.
- 4 Implement Winsorization (capping) on Annual_Income by setting extreme values exceeding upper and lower IQR boundaries directly to the respective threshold values, ensuring zero row deletions.

Q4 Web Scraping & Dynamic Dataset Integration

Task Objective

Scrape structured content from an external web endpoint, clean and transform the extracted data, and combine it with the primary CSV dataset records.

Tasks

- 1 Issue a programmatic HTTP GET request using `requests.get()` to retrieve HTML content from `https://quotes.toscrape.com/`.
- 2 Parse the HTML tree structure with BeautifulSoup using standard CSS class selectors (`.text`, `.author`, `.tag`).
- 3 Extract text elements into a structured DataFrame (`scraped_df`) containing columns: Quote, Author, and Tags.
- 4 Export the newly generated dataframe into a standalone CSV file named `scraped_data.csv` using `index=False`.
- 5 Join or merge `scraped_df` with the existing `df` (from `customer_data.csv`) using a synthetic index alignment or key grouping, and display the first 5 records of the merged DataFrame.

Q5 Bernoulli & Categorical Distributions (Discrete & Multi-Class Data)

Task Objective

Model binary and multi-class categorical features in `customer_data.csv` using the Bernoulli and categorical distributions, and validate distributional assumptions with a formal goodness-of-fit test.

Tasks

- 1 Load `customer_data.csv` and estimate the parameter p -hat (the proportion of churned customers) for the Bernoulli variable `Churn_Flag`.
- 2 Calculate the theoretical mean (p) and variance ($p(1-p)$) for `Churn_Flag` and compare them with the sample mean and sample variance.
- 3 Compute the empirical probability distribution across all categories in `Payment_Method`.
- 4 Perform a Chi-Square Goodness-of-Fit test using `scipy.stats.chisquare` to determine whether payment methods are uniformly distributed.
- 5 Plot the discrete distributions using Seaborn count plots with percentage annotations.

Q6 Binomial, Geometric & Poisson Distributions (Discrete Count Data)

Task Objective

Model repeated-trial and count-based processes drawn from `customer_data.csv` using the Binomial, Geometric, and Poisson distributions, and compare theoretical predictions with observed data.

Tasks

- 1 Using `customer_data.csv`, select an appropriate binary variable and estimate its success probability (p).
- 2 Model the number of successful outcomes in $n = 20$ independent trials using a Binomial distribution.
- 3 Calculate and plot the Binomial PMF for $X = 0, 1, \dots, 20$.
- 4 Calculate the probability that the first successful outcome occurs on the k -th trial using the Geometric distribution.
- 5 Identify a suitable count variable such as the number of customer interactions per day and estimate its average rate (λ).
- 6 Model the number of interactions occurring in a fixed time interval using a Poisson distribution and compare the theoretical probabilities with the empirical distribution.
- 7 Visualize the Binomial, Geometric, and Poisson PMFs using bar plots.

Q7 Uniform, Normal, Exponential & Gamma Distributions (Continuous Data)

Task Objective

Fit and evaluate several continuous probability distributions against numerical features of `customer_data.csv`, comparing theoretical densities with the empirical data.

Tasks

- 1 Select a continuous numerical variable such as Age, Monthly_Charges, or Tenure from `customer_data.csv`.
- 2 Calculate the sample mean, variance, and standard deviation.
- 3 Fit a Normal distribution using the estimated mean and standard deviation and compare the fitted PDF with the empirical histogram.
- 4 Standardize the selected variable using its Z-score and verify its relationship with the Standard Normal distribution.
- 5 Generate samples from a Uniform distribution over the observed minimum and maximum values and compare its density with the observed data.
- 6 Model waiting-time data using an Exponential distribution and estimate the rate parameter (λ).
- 7 Fit a Gamma distribution to a positive continuous variable and compare its PDF with the empirical distribution.
- 8 Plot histograms with KDE curves and fitted probability density functions for the selected distributions.

Q8 Beta, Log-Normal & Multivariate Normal Distributions (Probability & Multivariate Data)

Task Objective

Fit distributions suited to bounded proportions, right-skewed positive variables, and jointly distributed numerical pairs within `customer_data.csv`.

Tasks

- 1 Select a probability/proportion variable or transform an appropriate numerical variable to the interval $[0, 1]$ and fit a Beta distribution using `scipy.stats.beta`.
- 2 Estimate the Beta distribution parameters α and β and visualize the fitted PDF.
- 3 Select a positive, right-skewed variable such as `Monthly_Charges` and investigate whether a Log-Normal distribution provides a good fit.
- 4 Apply a logarithmic transformation and compare the transformed data with a Normal distribution using a histogram and Q-Q plot.
- 5 Select two correlated numerical variables, such as `Monthly_Charges` and `Total_Charges`, and estimate their mean vector and covariance matrix.
- 6 Model their joint behavior using a Multivariate Normal distribution.
- 7 Visualize the relationship using a scatter plot and compare the observed data with samples generated from the fitted Multivariate Normal distribution.

Q9 t, Chi-Square & F Distributions (Statistical Inference & Hypothesis Testing)

Task Objective

Apply classical inferential procedures built on the t, Chi-Square, and F distributions to test hypotheses about means, variances, independence, and group differences in `customer_data.csv`.

Tasks

- 1 Select a numerical variable from `customer_data.csv` and calculate its sample mean.
- 2 Perform a one-sample t-test using `scipy.stats.ttest_1samp` to determine whether the population mean differs significantly from a specified value.
- 3 Use the t-distribution to construct a confidence interval for the population mean.
- 4 Perform a Chi-Square test of independence between two categorical variables, such as `Churn_Flag` and `Payment_Method`, using `scipy.stats.chi2_contingency`.
- 5 Perform a Chi-Square Goodness-of-Fit test to determine whether observed categorical frequencies match an expected distribution.
- 6 Compare the variances of two numerical groups using an F-distribution-based test.
- 7 Perform an ANOVA test using `scipy.stats.f_oneway` to determine whether the means of a numerical variable differ significantly across multiple customer groups.
- 8 Report the test statistic, p-value, null hypothesis, alternative hypothesis, and final statistical conclusion.